

Rapport d'Audit de Sécurité — Projet E-commerce

Cours : DevSecOps **Groupe :** 7

Membres du groupe : * Dani Dias De Sousa * Adam Boukhajou * Quentin Deiss * Hippolite Valatta * Sofiane Dahbi

1. Introduction

Dans le cadre de notre audit de sécurité sur l'application E-commerce, nous avons analysé le code source (Backend, Frontend) ainsi que la configuration de l'infrastructure (Docker, CI). Ce document recense les vulnérabilités identifiées, classées par sévérité, et propose des correctifs pour sécuriser l'application.

L'audit met en évidence plusieurs failles critiques, notamment des risques d'exécution de code à distance et une mauvaise gestion des secrets, qui doivent être corrigées en priorité avant toute mise en production.

2. Analyse des Vulnérabilités : Frontend

2.1. Injection de Code via `eval()`

- **Gravité :** Critique
- **Fichier concerné :** `frontend/src/App.js` (Barre de recherche)
- **Description :** Le code utilise la fonction `eval()` pour filtrer les produits. Cette fonction exécute n'importe quelle chaîne de caractères comme du code JavaScript.
- **Impact :** Un attaquant peut injecter du code arbitraire s'exécutant directement sur le navigateur de l'utilisateur (XSS, vol de session, redirection).
- **Preuve de Concept (PoC) :** En entrant `'); alert('XSS'); ('` dans la barre de recherche, le code injecté est exécuté.
- **Correction recommandée :** Ne jamais utiliser `eval()`. Il faut utiliser les méthodes natives de JavaScript pour filtrer les tableaux.
`javascript // Correction`
`return p.name.toLowerCase().includes(searchQuery.toLowerCase());`

2.2. Cross-Site Scripting (XSS) via `dangerouslySetInnerHTML`

- **Gravité :** Élevée
- **Fichiers concernés :** `frontend/src/App.js` (Affichage produits et commentaires)
- **Description :** L'application utilise la propriété React `dangerouslySetInnerHTML` pour afficher le nom des produits et les commentaires sans nettoyage préalable.

- **Impact :** Si un produit ou un commentaire contient du code HTML/JS malveillant (injecté en base de données), ce code s'exécutera chez tous les utilisateurs qui consultent la page (XSS Stocké).
- **Correction recommandée :** Utiliser une bibliothèque comme **DOMPurify** pour nettoyer le HTML avant l'affichage, ou afficher les données en texte brut si le HTML n'est pas nécessaire.

2.3. Collecte de données bancaires non sécurisée

- **Gravité :** Critique
- **Fichier concerné :** frontend/src/App.js
- **Description :** Le numéro de carte bancaire est demandé via un simple prompt() JavaScript et manipulé en clair par l'application.
- **Impact :** Violation majeure des normes de sécurité. Risque critique de vol de données bancaires.
- **Correction recommandée :** Supprimer cette méthode de collecte. Intégrer un prestataire de paiement sécurisé comme Stripe ou PayPal qui gère la saisie via des iframes sécurisées (tokenisation).

2.4. Stockage non sécurisé des Tokens (LocalStorage)

- **Gravité :** Élevée
- **Fichier concerné :** frontend/src/App.js
- **Description :** Le token d'authentification (JWT) est stocké dans le localStorage du navigateur. De plus, il est affiché dans les logs de la console via console.log.
- **Impact :** En cas de faille XSS, un attaquant peut facilement voler le token et usurper l'identité de l'utilisateur.
- **Correction recommandée :**
 - Stocker le token dans un **Cookie HttpOnly** (inaccessible via JavaScript) avec les attributs Secure et SameSite.
 - Supprimer tous les console.log affichant des données sensibles.

2.5. Absence de validation des formulaires (Inscription / Login)

- **Gravité :** Élevée
- **Fichiers concernés :** frontend/src/App.js
- **Description :**

Les formulaires reposent uniquement sur les attributs HTML (required, type="email", type="password").

Ces attributs peuvent être supprimés depuis les DevTools, permettant d'envoyer des champs vides ou invalides.

- **Impact :**
Création de comptes avec des valeurs incorrectes, injections possibles, contournement total des contrôles côté client.
- **PoC :**
Suppression de required dans l'inspecteur → le formulaire accepte un email invalide ou vide.
- **Correction recommandée :**
 - Ajouter une validation JavaScript et surtout valider les données côté backend.
 - Implémenter une validation stricte des paramètres côté backend (email conforme, mot de passe fort, champs non vides).
 - Ajouter des validations frontend robustes (Regex, fonctions JS, bibliothèques comme Yup ou Validator.js).
 - Ne jamais se reposer sur les attributs HTML pour garantir la sécurité.
 - Retourner un message d'erreur clair en cas d'entrée invalide.

2.6. Absence de politique de mot de passe (Inscription)

- **Gravité :** Moyenne à Élevée
- **Fichiers concernés :** frontend/src/App.js
- **Description :**
Le formulaire d'inscription accepte des mots de passe très faibles (ex. a, 123, password) sans aucun contrôle de longueur ni de complexité côté frontend.
- **Impact :**
Les utilisateurs peuvent choisir des mots de passe triviales, ce qui facilite les attaques par brute force ou par dictionnaire sur les comptes.
- **PoC :**
Création d'un compte avec un mot de passe extrêmement simple (a) acceptée sans avertissement.
- **Correction recommandée :**
Ajouter une validation côté frontend (et surtout backend) imposant une longueur minimale et un mot de passe composé de lettres, chiffres et caractères spéciaux.

2.7. Absence de protection contre la force brute (Formulaire de connexion)

- **Gravité :** Élevée
- **Fichiers concernés :** frontend/src/App.js

- **Description :**
Le formulaire de connexion permet un nombre illimité de tentatives sans délai, sans verrouillage de compte ni mécanisme de protection (captcha, temporisation, etc.).
- **Impact :**
Un attaquant peut automatiser des tentatives de connexion massives (brute force ou dictionnaire) depuis l'interface publique sans rencontrer de blocage côté client.
- **PoC :**
Enchaîner plusieurs dizaines de tentatives de connexion avec des identifiants aléatoires depuis le formulaire ne déclenche aucune limite ni message spécifique.
- **Correction recommandée :**
Mettre en place côté backend une limitation de tentatives (rate limiting, verrouillage temporaire du compte) et côté frontend un message d'erreur générique après plusieurs échecs.

2.8. Déni de service côté client via manipulation du DOM (crash de l'interface)

Gravité : Élevée

Fichier concerné : frontend/src/App.js **Description :**

La fonction `eval()` utilisée dans la barre de recherche exécute directement le contenu saisi par l'utilisateur.

En injectant du JavaScript qui modifie le DOM, il est possible de rendre l'interface totalement inutilisable.

Une simple recherche peut donc supprimer l'intégralité de la page et provoquer un crash de l'interface React. **Impact :**

- Déni de service côté client : la page devient entièrement blanche.
- L'utilisateur ne peut plus naviguer ni interagir avec l'interface.
- Le site reste inutilisable tant qu'il n'est pas rechargé (et parfois nécessite un nettoyage du stockage local selon le payload).
- Un attaquant peut provoquer cette situation de manière triviale. **Preuve de Concept (PoC) :**

(PoC) :

En entrant le payload suivant dans la barre de recherche :

```
' ); document.body.innerHTML = ""; //
```

et en cliquant sur *Rechercher*, toute l'interface disparaît instantanément. Le DOM est vidé et la page reste blanche. **Correction recommandée :**

- Supprimer totalement l'usage de `eval()`.
- Valider et filtrer strictement les entrées utilisateur.
- Encadrer les opérations critiques dans des blocs `try/catch`.
- Empêcher toute exécution directe de code provenant de l'utilisateur.

2.9. Authentification cassée (Login toujours en échec)

- **Gravité :** Élevée
- **Fichiers concernés :** frontend/src/App.js, backend/routes/auth.js

- **Description :**
Lors de l'inscription, l'utilisateur est correctement ajouté en base de données (code HTTP 204).
Cependant, toute tentative de connexion échoue systématiquement avec une réponse **401 Unauthorized**.
Le frontend recharge la page au lieu d'afficher un message d'erreur, ce qui rend la panne invisible pour l'utilisateur.
- **Preuve (Network DevTools) :**
 - POST /register → **204** (succès)
 - POST /login → **401 Unauthorized** (échec, même avec les bons identifiants)
- **Causes possibles :**
 - Le mot de passe est **hashé lors de l'inscription**, mais envoyé **en clair lors de la connexion**, rendant `bcrypt.compare()` toujours faux.
 - Le token généré pendant l'inscription est stocké dans le `localStorage`, mais **jamais utilisé ni vérifié** dans la route de login.
 - Le frontend n'affiche aucun message d'erreur et effectue simplement un refresh de la page.
- **Impact :**
 - Connexion totalement impossible, même avec un compte valide.
 - Perte complète de la fonctionnalité d'authentification.
 - Mauvaise expérience utilisateur liée à l'absence de feedback.
 - Possibilité de déni de service logique en bloquant les connexions.
- **Correction recommandée :**
 - Harmoniser la gestion des mots de passe :
 - Hashage à l'inscription
 - `bcrypt.compare()` correctement implémenté lors du login
 - Supprimer le refresh automatique et afficher des erreurs claires côté frontend.
 - Ne plus stocker le token en clair dans le `localStorage` (préférer un cookie `HttpOnly`).
 - Ajouter une validation stricte des entrées côté backend.

2.10. Clé API Stripe codée en dur dans le frontend

Gravité : Critique

Fichier concerné : frontend/src/App.js

Description : Le code frontend contient une clé API Stripe « live » directement codée en dur : `const API_KEY = 'sk_live_41Hqp9K2eZvKYlo2C8xO3n4y5z6a7b8c9d0e1f2g3h4i5p';`

Toute personne ayant accès au code ou au bundle JavaScript peut récupérer cette clé.

Impact : Un attaquant peut réutiliser la clé Stripe pour : - initier des paiements frauduleux, - interagir avec l'API Stripe au nom de l'application, - compromettre entièrement le compte Stripe lié.

Correction recommandée : - Ne jamais mettre de clé secrète dans le frontend. - Utiliser des variables d'environnement côté backend uniquement, et exposer au mieux une clé « publishable » côté client si nécessaire. - Toutes les opérations sensibles doivent passer par le serveur (backend) qui lui seul connaît la vraie clé secrète.

2.11. IDOR côté frontend sur la récupération d'utilisateur

Gravité : Élevée

Fichiers concernés : frontend/src/App.js

Description : Le frontend permet de récupérer les informations d'un utilisateur en fonction d'un identifiant saisi librement : `const userId = document.getElementById('userId').value;`
`const response = await fetch(`${API_URL}/users/${userId});`

Si le backend n'applique pas de contrôle d'accès strict, l'utilisateur connecté peut modifier le `userId` et accéder aux données d'autres comptes.

Impact : - Accès non autorisé aux informations d'autres utilisateurs. - Possibilité de consulter des comptes sensibles (ex : compte admin). - Couplé avec le backend vulnérable (`/api/users/:id`), cela permet une fuite complète des données utilisateurs.

Preuve de Concept (PoC) : Saisir un autre identifiant (ex : 1, 2, 3...) dans le champ `userId` puis appeler l'API. Si les données d'un autre utilisateur sont renvoyées, l'IDOR est confirmée.

Correction recommandée : Côté frontend, appeler un endpoint de type `/users/me` plutôt que `/users/:id`. Côté backend, vérifier systématiquement que l'utilisateur ne peut accéder qu'à ses propres informations, via le token JWT ou la session, sans jamais se baser sur un id fourni par le client.

2.12. Dépendance à une ressource externe pour le background

Gravité : Faible à Moyenne

Fichier concerné : frontend/src/index.css

Description : L'application charge une image de fond directement depuis un domaine externe (Unsplash) : `background: url('https://images.unsplash.com/photo-1441986300917-64674bd600d8?w=1920&q=80') ...;`

Le site dépend donc d'un fournisseur tiers pour l'affichage, et le contenu pourrait être modifié côté serveur externe sans contrôle de l'application.

Impact : – Risque de tracking ou de collecte de métadonnées via la ressource externe. – Dépendance de disponibilité : si le domaine externe est indisponible, l'interface peut être dégradée. – En cas de compromission de la ressource, possibilité d'afficher un contenu inapproprié ou malveillant.

Correction recommandée : Héberger l'image en local dans l'application (ex : `/assets/background.jpg`) et référencer uniquement des ressources contrôlées par l'équipe : `background: url('/assets/background.jpg') ...;`

3. Analyse des Vulnérabilités : Backend

3.1. Exécution de Code à Distance via `eval()`

- **Gravité :** Critique
- **Fichier concerné :** `server.js` (Endpoint `/api/products/search`)
- **Description :** Comme sur le frontend, le backend utilise `eval()` pour traiter les requêtes de recherche.
- **Impact :** C'est la faille la plus dangereuse. Elle permet à un attaquant de prendre le contrôle total du serveur (lecture de fichiers système, suppression de données, installation de malwares).
- **Correction recommandée :** Remplacer la logique de recherche par une requête sécurisée en base de données ou l'utilisation de méthodes de filtrage natives sans exécution dynamique.

3.2. Exposition totale des variables d'environnement

- **Gravité :** Critique
- **Endpoint :** `/api/debug`
- **Description :** Une route de débogage renvoie l'intégralité de l'objet `process.env`.
- **Impact :** Fuite immédiate de tous les secrets : clés API, secrets JWT, identifiants de base de données. Cela compromet toute l'infrastructure.
- **Correction recommandée :** Supprimer immédiatement et définitivement la route `/api/debug` du code source.

3.3. Contournement d'authentification (SQL Injection logic)

- **Gravité :** Élevée
- **Fichier concerné :** `server.js` (`/api/login`)
- **Description :** Le code contient une condition explicite permettant le bypass d'authentification si le nom d'utilisateur contient `' OR '1'='1'`. De plus, les mots de passe sont en clair.
- **Correction recommandée :**
 - Retirer la logique de bypass.

- Hacher les mots de passe avec un algorithme robuste (ex: **bcrypt**) au lieu de les stocker/comparer en clair.

3.4. Path Traversal (Lecture de fichiers arbitraires)

- **Gravité** : Moyenne/Élevée
- **Endpoint** : `/api/files/:filename`
- **Description** : L'API lit des fichiers basés sur l'entrée utilisateur sans vérifier si le chemin remonte dans l'arborescence (ex: `../../.env`).
- **Correction recommandée** : Nettoyer le nom du fichier (utiliser `path.basename`) et vérifier que le chemin résolu reste strictement dans le dossier autorisé (`uploads/`).

3.5. Exposition complète des utilisateurs via l'endpoint `/api/users`

Gravité : Critique

Endpoint concerné : `/api/users` Description :

L'API expose publiquement la route `/api/users` sans aucune authentification. Cette route renvoie l'intégralité de la base utilisateurs, incluant des informations extrêmement sensibles telles que les mots de passe en clair, les emails, les rôles, et même des clés API ou des numéros de carte bancaire selon les comptes. **Impact :**

Cette vulnérabilité permet à n'importe quel attaquant d'accéder aux informations des utilisateurs, y compris l'administrateur.

Elle permet également de se connecter directement aux comptes, car les mots de passe sont stockés en clair dans la base de données.

Plus globalement, cela entraîne une compromission totale de l'application. **Preuve de**

Concept (PoC) :

En accédant simplement à l'URL :

`http://localhost:5001/api/users`

Le serveur renvoie des données comme :

```
[
  {"id":1,"username":"admin","password":"admin123","email":"admin@ecommerce.com","role":"admin","apiKey":"admin-key-123456"},
  {"id":2,"username":"user","password":"user123","email":"user@example.com","role":"customer","creditCard":"4532-1234-5678-9010"},
  {"id":3,"username":"test","password":"test","email":"test@gmail.com","role":"customer"}
]
```

Aucune authentification n'est requise, ce qui permet de consulter et d'utiliser directement les identifiants. **Correction recommandée** : - Protéger la route `/api/users` avec une authentification stricte et une vérification du rôle admin.

- Ne jamais renvoyer les mots de passe (même hashés) ni les clés API dans les réponses API.

- Filtrer les données sensibles avant l'envoi (whitelisting des champs).
- Mettre en place une gestion des permissions robuste sur toutes les routes.

3.6. IDOR : Accès direct aux comptes via /api/users/:id

Gravité : Critique Endpoint concerné : /api/users/:id Description : L'API permet d'accéder aux informations d'un utilisateur simplement en modifiant l'ID dans l'URL, sans aucune authentification ni contrôle d'accès. Par exemple :

```
GET /api/users/1
GET /api/users/2
GET /api/users/3
```

Ces requêtes renvoient directement les données complètes de chaque utilisateur : - username - password (en clair) - email - rôle - apiKey - autres données sensibles ### Impact : - Accès total à tous les comptes utilisateurs. - Possibilité de récupérer tous les mots de passe en clair. - Connexion directe aux comptes de n'importe quel utilisateur. - Compromission immédiate du compte administrateur. - Prise de contrôle complète de l'application. ### Preuve : En accédant simplement à : http://localhost:5001/api/users/1

La réponse renvoyée est :

```
{id":1,"username":"admin","password":"admin123","email":"admin@ecommerce.com",
"role":"admin","apiKey":"admin-key-123456"}
```

Correctif recommandé :

- Protéger la route avec un système d'authentification.
- Ajouter un contrôle d'accès : un utilisateur ne doit voir que ses propres données.
- Ne jamais renvoyer les mots de passe.
- Ne jamais renvoyer de clés API.
- Stocker les mots de passe en hash (bcrypt).

3.7. Contournement total de l'authentification (Login) via payload malveillant

Gravité : Critique Endpoint concerné : /api/login Description : Le backend contient une condition volontairement vulnérable dans le code de l'endpoint /api/login :

```
if (username.includes("' OR '1'='1')) {
  return res.json({
    success: true,
    token: jwt.sign({ username: "admin", role: "admin" }, SECRET),
    user: adminUser
  });
}
```

Cette condition permet à n'importe quel attaquant d'obtenir un jeton JWT administrateur simplement en envoyant un nom d'utilisateur contenant ' OR '1'='1'. Aucune vérification du mot de passe n'est effectuée lorsque cette condition est vraie.

Preuve de Concept (PoC) *Requête envoyée:* POST /api/login Content-Type: application/json

```
{"username": "' OR '1'='1", "password": ""}
```

L'API renvoie donc un JWT admin utilisable, l'intégralité des données du compte admin (mot de passe en clair, email, clé API...).

Impact: - Contournement total de l'authentification. - Accès immédiat au compte administrateur, sans mot de passe. - Fuite critique : mots de passe en clair, email admin, clé API. - Possibilité d'accéder à toutes les fonctionnalités sensibles (gestion des produits, données des utilisateurs, etc.). - Compromission totale et instantanée de l'application.

Correction recommandée: - Supprimer immédiatement la condition permettant le contournement. - Valider strictement les données entrantes. - Hacher les mots de passe (bcrypt). - Ne jamais renvoyer les mots de passe ou clés API dans les réponses JSON. - Mettre en place une authentification robuste et des contrôles d'accès stricts.

3.9. Absence d'authentification sur l'API Produits

Gravité : Élevée

Endpoint : /api/products **Description :**

L'API renvoie l'intégralité des produits **sans aucune authentification**.

N'importe quel utilisateur, même non connecté, peut accéder aux informations internes du catalogue (prix, stock, catégories).

PoC :

```
curl http://localhost:5001/api/products
```

Impact : - Extraction complète du catalogue (scraping). - Analyse concurrentielle / copie du catalogue. - Exploitation possible pour attaques plus avancées.

Correctif : Exiger un token JWT valide avant d'autoriser l'accès aux produits.

3.10. CORS mal configuré (Autorise toutes les origines)

Gravité : Élevée **Fichier concerné :** server.js

Description : Le backend utilise un CORS permissif :

```
app.use(cors());
```

Cela autorise tous les domaines externes (Access-Control-Allow-Origin: *). N'importe quel site Web peut effectuer des requêtes à l'API au nom de l'utilisateur connecté (si le token ou la session est active).

Impact : - Vol de session via requêtes cross-site - Actions non autorisées envoyées depuis un site malveillant - Contournement des protections navigateur - Exécution d'actions critiques sur l'API à l'insu de l'utilisateur

PoC : Depuis n'importe quel site externe :

```
fetch("http://localhost:5001/api/users")
  .then(r => r.json())
  .then(console.log);
```

Correctif recommandé : Limiter les origines autorisées

```
app.use(cors({
  origin: "http://localhost:3000",
  credentials: true
}));
```

3.11. Absence totale de rate limiting

Gravité : Élevée

Description : Toutes les routes (login, register, users, cart...) acceptent un nombre illimité de requêtes.

Impact : Attaques par force brute, déni de service.

Correctif : Ajouter express-rate-limit. ### 3.12. Pas de validation de données côté backend **Gravité :** Critique

Description : Les routes acceptent n'importe quelle entrée sans validation (email, user, produits...).

Impact : injections, crash serveur, création d'objets invalides.

Correctif : Utiliser Joi / Zod / express-validator. ### 3.12. Absence totale de logs de sécurité **Gravité :** Moyenne

Description : Le backend ne logge ni les tentatives ratées ni les accès sensibles.

Impact : Impossible de détecter une attaque.

Correctif : Ajouter Winston / Morgan avec logs structurés. ### 3.13. Sessions non sécurisées (cookie non HttpOnly, non Secure) **Gravité :** Élevée

Description : Le cookie connect.sid n'a pas les flags HttpOnly/Secure/SameSite.

Impact : Vol de session via JavaScript ou XSS.

Correctif : Activer HttpOnly: true, secure: true.

3.14. Mauvaise gestion des erreurs (stack exposée)

Gravité : Moyenne

Description : Les erreurs renvoient des stack traces Node.js complètes.

Impact : Fuite d'informations sensibles.

Correctif : Masquer les stacks en production.

3.15. API Key envoyée dans chaque requête (header x-api-key)

Gravité : Élevée

Description : Le frontend ajoute une clé "x-api-key" publique dans toutes les requêtes.

Impact : Rejeu de requêtes, contournement d'authentification.

Correctif : Supprimer et gérer les accès via JWT.

3.16. Manque de séparation des rôles (pas de RBAC)

Gravité : Critique

Description : Admin / user / guest ont accès aux mêmes routes.

Impact : Escalade horizontale et verticale.

Correctif : Ajouter middleware de permission par rôle.

3.17. Headers de sécurité absents

Gravité : Moyenne

Description : Pas de X-Frame-Options, CSP, HSTS, X-XSS-Protection.

Impact : Vulnérabilités Clickjacking / XSS / MITM.

Correctif : Utiliser `helmet()`.

3.18. Passwords stockés en clair dans la base

Gravité : Critique

Description : Aucun hachage Bcrypt. Les mots de passe sont lisibles.

Impact : Fuite totale de tous les comptes.

Correctif : `bcrypt.hash()` + salage.

3.19. Manque d'isolation Docker

Gravité : Moyenne

Description : L'app tourne en root dans le conteneur.

Impact : Évasion de conteneur possible.

Correctif : Ajouter `USER node` dans le Dockerfile.

3.20. Accès direct à MongoDB sans authentification

Gravité : Critique

Description : Le docker-compose lance MongoDB sans mot de passe.

Impact : Un attaquant se connecte en root → suppression des données.

Correctif : Activer auth + variables d'environnement sécurisées.

3.21. Pas de limitation de taille des requêtes

Gravité : Élevée

Description : `express.json()` accepte des payloads illimités.

Impact : DoS via body de 100MB.

Correctif : `express.json({ limit: "1mb" })`.

3.22. Absence de CSRF protection

Gravité : Élevée

Description : Les actions sensibles (login, users, cart) ne nécessitent aucun token CSRF.

Impact : Un site externe peut exécuter des actions à la place de l'utilisateur.

Correctif : Ajouter `csrf()`.

3.23. Pas de sanitation des inputs HTML

Gravité : Élevée

Description : Le backend stocke les reviews et descriptions sans nettoyage.

Impact : XSS persistante dans la base.

Correctif : Nettoyer avec DOMPurify côté backend.

4. Infrastructure (Docker & CI)

4.1. Secrets codés en dur (Hardcoded Secrets)

- **Gravité :** Élevée
- **Fichiers concernés :** Dockerfile, docker-compose.yml
- **Description :** Les clés API et secrets (JWT_SECRET) sont écrits en clair directement dans les fichiers de configuration Docker.
- **Impact :** Toute personne ayant accès au dépôt Git ou à l'image Docker peut récupérer ces clés.
- **Correction recommandée :** Utiliser des variables d'environnement injectées au moment du déploiement (via un gestionnaire de secrets ou .env non versionné). Ne jamais commiter de secrets sur Git.

4.2. Gestion des images Docker

- **Gravité :** Moyenne
- **Description :** Les Dockerfiles utilisent des images de base génériques (ex: node:16) ou obsolètes, et la CI utilise des actions GitHub sans versionnage précis (@master).
- **Correction recommandée :**
 - Utiliser des images légères et versionnées, par exemple : node:20.11.0-alpine.

Analyse du pipeline .github/workflows/security.yml

Dans ce projet, le fichier .github/workflows/security.yml met en place un **pipeline DevSecOps complet** qui se déclenche automatiquement à chaque push et pull_request. L'objectif est d'automatiser les contrôles de sécurité sur le code, les dépendances et les images Docker avant intégration.

1) Structure générale du pipeline

Le pipeline est découpé en plusieurs jobs :

1. build-backend

- Récupère le code (actions/checkout@v4).

- Construit l'image Docker du backend : `docker build -t ecommerce-backend:${{ github.sha }} ./backend`.
- Sauvegarde l'image dans un fichier `backend-image.tar` et l'upload comme artefact pour les autres jobs.

2. build-frontend

- Même logique, mais pour le frontend.
- Construit l'image `ecommerce-frontend:${{ github.sha }}` puis l'upload dans `frontend-image.tar`.

Ces deux premiers jobs préparent les images pour les scans de sécurité (Trivy).

2) SAST – Analyse statique du code (sast)

- Job **sast** : analyse du **code source** avec **Semgrep**.
- Il :
 - installe Semgrep,
 - scanne le dépôt avec un set de règles (type OWASP Top 10),
 - remonte les problèmes de sécurité dans le code (XSS, injections, mauvaise gestion d'auth, eval, etc.).

Ce job permet de vérifier que les corrections dans `server.js`, `App.js`, etc. sont conformes aux bonnes pratiques et qu'aucune nouvelle faille n'a été introduite.

3) SCA – Analyse des dépendances (sca-backend / sca-frontend)

Deux jobs distincts :

1. sca-backend

- Installe les dépendances du backend (`npm install`).
- Lance `npm audit --json > audit-backend.json`.
- Sauvegarde le rapport comme artefact.

2. sca-frontend

- Même logique mais dans `./frontend`.

- Génère audit-frontend.json.

Ces jobs font de la **Software Composition Analysis (SCA)** :

ils détectent les vulnérabilités connues dans les librairies npm utilisées (CVE, dépendances obsolètes, etc.).

Le but est de réduire au maximum les vulnérabilités **HIGH/CRITICAL** et de garder les dépendances à jour.

4) Détection de secrets dans le repo (secrets)

- Job **secrets** : utilise **Gitleaks**.
- Il scanne le dépôt Git pour détecter :
 - clés API,
 - tokens,
 - mots de passe,
 - clés JWT ou secrets committés par erreur.

Ce job complète le travail que tu as fait :

les secrets sont maintenant dans .env / variables d'environnement et plus en dur dans le code ou dans docker-compose.yml.

5) Scans de conteneurs avec Trivy

Trois jobs :

1. container-scan-backend

- Télécharge l'artefact backend-image.
- Charge l'image Docker : docker load < backend-image.tar.
- Lance **Trivy** sur l'image backend avec severity: CRITICAL et exit-code: 1.
→ Si une vulnérabilité critique est trouvée dans l'image backend, le job échoue.

2. container-scan-frontend

- Même principe pour l'image frontend.

3. container-scan-mongodb

- Scanne directement l'image officielle mongo:5.0 avec Trivy.
→ Permet de voir les CVE présentes dans l'image de base (important pour le choix des versions).

Ces jobs garantissent que **les images Docker utilisées en déploiement ne contiennent pas de vulnérabilités critiques**.

6) Job de synthèse : summary

Le job **summary** dépend de tous les autres (needs: ...).

Il :

- récupère les résultats de tous les jobs de sécurité (SAST, SCA, secrets, container scans),
- construit un **résumé JSON** avec le statut de chaque contrôle,
- écrit un compte-rendu lisible dans \$GITHUB_STEP_SUMMARY :
 - si au moins un job a échoué → message  "Security issues detected!" + échec du pipeline ;
 - si tout est OK → message  "All security checks passed".

Ce job sert de **point central de décision** :

tant qu'une analyse échoue, le pipeline bloque, ce qui empêche la mise en production d'une version vulnérable.

7) Conclusion : en quoi ce pipeline est DevSecOps ?

Ce pipeline illustre bien l'approche **DevSecOps** :

- **Dev** : construction automatique des images backend/frontend (build-backend / build-frontend).
- **Sec** :
 - SAST avec Semgrep sur le code,
 - SCA avec npm audit sur les dépendances,
 - scan de secrets avec Gitleaks,
 - scans d'images Docker avec Trivy (backend, frontend, MongoDB).

- **Ops :**
 - utilisation de Docker,
 - artefacts réutilisables entre jobs,
 - résumé automatique de l'état de sécurité du projet.

Les contrôles de sécurité sont exécutés **à chaque push / pull request**, donc **très tôt dans le cycle de développement** ("shift-left security").

Le pipeline ne remplace pas l'analyse humaine (IDOR, logique métier, etc.), mais il automatise une grosse partie des vérifications de base et garantit qu'aucune version avec des vulnérabilités critiques ne passe sans être détectée.

5. Conclusion

Ce projet DevSecOps m'a permis de comprendre et de mettre en pratique les bonnes pratiques de sécurité tout au long du cycle de développement d'une application web. L'analyse du code, la détection des vulnérabilités et la mise en place d'un pipeline CI/CD sécurisé ont montré l'importance d'intégrer la sécurité dès les premières étapes ("shift-left security").

Grâce aux outils comme Semgrep, npm audit, Gitleaks et Trivy, j'ai pu identifier et corriger les failles présentes dans le backend, le frontend et même dans les images Docker. L'utilisation de conteneurs sécurisés, d'un utilisateur non-root, de variables d'environnement et d'un docker-compose propre renforce encore la robustesse de l'application.

Au final, l'application a été entièrement revue, sécurisée et validée par le pipeline. Cette approche m'a montré que la sécurité n'est pas une étape finale, mais un processus continu intégré au développement. Ce projet m'a donné une vision concrète et complète du rôle essentiel du DevSecOps dans un environnement moderne de développement logiciel.